

ASSIGNMENT 8

Q1: Explain Data-Driven Testing in Selenium. How can Apache POI be used to read and write data in Excel for automation testing? Provide an example scenario.

Data-Driven Testing (DDT) is a testing approach where test data is stored outside the test script (such as Excel, CSV, or Database). The same Selenium test script is executed multiple times using different input values.

Advantages

- Reduces duplicate code.
- Easy to maintain test data.
- Improves test coverage.
- Supports multiple test scenarios.

Apache POI

Apache POI is a Java library used to read and write Microsoft Excel (.xlsx/.xls) files.

It can:

- Read username and password from Excel.
- Write test results (Pass/Fail) back into Excel.
- Use different test data without modifying the code.

Example Scenario

Suppose an Excel sheet contains:

Username	Password
admin	admin123
user1	test123
user2	abc123

The Selenium script:

1. Reads each username and password using Apache POI.
2. Performs login.
3. Verifies the result.
4. Writes Pass or Fail into another Excel column.

Q2: Write a Selenium WebDriver + POM (Page Object Model) based script in Java for a Login Page.

Program:

- LoginPage.java (Page Object):

```
package Pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class LoginPage {

    WebDriver driver;

    By username = By.id("username");
    By password = By.id("password");
    By loginBtn = By.id("submit");

    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }

    public void enterUsername(String user) {
        driver.findElement(username).sendKeys(user);
    }

    public void enterPassword(String pass) {
        driver.findElement(password).sendKeys(pass);
    }

    public void clickLogin() {
        driver.findElement(loginBtn).click();
    }

    public void login(String user, String pass) {
        enterUsername(user);
        enterPassword(pass);
    }
}
```

```
        clickLogin();
    }
}
```

- LoginTest.java (Test Execution):

```
package Pages;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;

public class LoginTest {
    WebDriver driver;

    @BeforeMethod
    public void setup() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.get("https://practicetestautomation.com/practice-test-login/");
    }

    @Test
    public void loginTest() {
        LoginPage lp = new LoginPage(driver);
        lp.login("student", "Password123");
    }
}
```

Output:

PASSED: Pages.LoginTest.loginTest

Q3: What is the role of a TestNG.xml file? Write a sample testng.xml configuration for running 2 test classes in parallel.

The **testng.xml file** is used to control the execution of TestNG test cases.

Role of testng.xml

- Executes multiple test classes.
- Runs tests in parallel.
- Groups test cases.
- Passes parameters.
- Controls execution order.
- Generates reports.

Sample testng.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="Suite" parallel="classes">
  <test thread-count="2" name="LoginAndCheckoutTests">
    <classes>
      <class name="Tstng.LoginTest"/>
      <class name="Tstng.CheckoutTest"/>
    </classes>
  </test> <!-- Test -->
</suite> <!-- Suite -->
```

Q4: Explain the attributes of @Test annotation in TestNG with examples for:

priority

dependsOnMethods

enabled

1. Priority

- Determines the execution order of test methods.
- Lower priority executes first.

Example:

```
@Test(priority = 1)
public void login() {
    System.out.println("Login");
}

@Test(priority = 2)
public void search() {
    System.out.println("Search");
}
```

Output:

Login

Search

2. dependsOnMethods

- Runs a test only after another test executes successfully.

Example:

```
@Test
public void login() {
    System.out.println("Login Successful");
}

@Test(dependsOnMethods = "login")
```

```
public void logout() {  
    System.out.println("Logout Successful");  
}
```

- If login() fails, logout() will be skipped.

3. enabled

- Controls whether a test method should execute.

Example:

```
@Test(enabled = false)  
public void registration() {  
    System.out.println("Registration");  
}
```

- This test method will not execute

Q5: Create a Maven Project for Selenium. Write the sample POM.xml dependencies required for Selenium and TestNG, and explain how to execute test cases using Maven build tool.

pom.xml:

```
project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.testing</groupId>

    <artifactId>SeleniumMavenDemo</artifactId>

    <version>0.0.1-SNAPSHOT</version>

    <dependencies>

        <!-- Source: https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->
        <dependency>

            <groupId>org.seleniumhq.selenium</groupId>

            <artifactId>selenium-java</artifactId>

            <version>4.45.0</version>

            <scope>compile</scope>

        </dependency>

        <!-- Source: https://mvnrepository.com/artifact/org.testng/testng -->
        <dependency>

            <groupId>org.testng</groupId>

            <artifactId>testng</artifactId>

            <version>7.12.0</version>

            <scope>test</scope>

        </dependency>

    </dependencies>

</project>
```

testng.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="Suite">
    <test thread-count="5" name="Test">
        <classes>
            <class name="Assignment8.LoginTest"/>
        </classes>
    </test> <!-- Test -->
</suite> <!-- Suite -->
```

Test Class:

```
package Assignment8;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;

public class LoginTest {
    WebDriver driver;

    @BeforeMethod
    public void setup() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.get("https://practicetestautomation.com/practice-test-login/");
    }
}
```

@Test

```
public void titleTest() {  
    String title = driver.getTitle();  
    System.out.println("Page Title : " + title);  
}
```

@AfterMethod

```
public void close() {  
    driver.quit();  
}  
}
```

Output:

Page Title : Test Login | Practice Test Automation

PASSED: Assignment8.LoginTest.titleTest

Steps to Execute Test Cases Using Maven:

1. Create a Maven project in Eclipse.
2. Add Selenium and TestNG dependencies in pom.xml.
3. Save the file and allow Maven to download the required libraries.
4. Place the test classes in the src/test/java folder.
5. Run the Maven command:
 - mvn clean test: Cleans the project and then runs all tests.